Justificación | API de avistamientos

1. Cambios en el código

- Extraje la validación a la clase **ValidadorDeAvistamiento**, dejando al controlador únicamente la responsabilidad de recibir la petición y generar la respuesta HTTP. Con esto apliqué el principio de responsabilidad única (SRP) y pude probar la validación de forma aislada.

- Encapsulé la fecha actual en la interfaz Reloj, implementada por **RelojDelSistema** en producción. El validador recibe esta dependencia por constructor en lugar de llamar directamente a **LocalDate.now()**, aplicando el principio de inversión de dependencias (DIP).

- No fue necesario refactorizar el repositorio, ya que **AvistamientoRepository** ya se recibía mediante inyección de dependencias. Esto me permitió reemplazarlo por un mock en las pruebas sin modificar el código de producción.

Estos cambios mejoran la testeabilidad porque permiten reemplazar dependencias reales por mocks o stubs y probar cada componente de forma aislada. Por ejemplo, al usar un reloj con una fecha fija puedo controlar qué día es "hoy" y evitar que las pruebas dependan de la fecha real del sistema.

2. Tabla de casos parte A

Diseñé los casos de prueba aplicando partición de equivalencia y valores límite: primero identifiqué las particiones válidas e inválidas de cada campo y luego elegí un valor representativo de cada una, más los valores justo en los bordes de cada rango, que es donde suelen estar los errores. Para que las pruebas de fecha fueran repetibles, fijé la fecha de hoy en 2026-10-27 mediante un stub (RelojFijo).

2.1. Particiones de equivalencia

Campo	Partición	Rango	¿Válida?
Especie	E1	Vacía o solo espacios	No
	E2	1 a 2 caracteres	No
	E3	3 a 50 caracteres	Sí
	E4	51 o más caracteres	No
Fecha	F1	Vacía	No
	F2	Formato distinto de AAAA-MM-DD	No
	F3	Fecha inexistente	No
	F4	Fecha pasada o igual a hoy	Sí
	F5	Fecha posterior a hoy	No

2.2. Casos de prueba

#	Campo	Valor	Partición	Tipo de caso	Resultado esperado	Por qué este valor
1	especie	"guacamaya" (9)	E3	Representativo	Válido	Valor típico válido
2	especie	"abc" (3)	E3	Límite	Válido	Mínimo permitido, detecta si se escribió < 4 en vez de < 3
3	especie	"ab" (2)	E2	Límite	Error de especie	Debajo del mínimo
4	especie	50 caracteres	E3	Límite	Válido	Máximo permitido, detecta si se escribió >= 50 en vez de > 50
5	especie	51 caracteres	E4	Límite	Error de especie	Encima del máximo
6	especie	" "	E1	Inválido	Falta especie	Solo espacios
7	especie	" ab "	E2	Límite	Error de especie	Verifica trim()
8	fecha	"2026-09-15"	F4	Representativo	Válido	Fecha pasada válida
9	fecha	"2026-10-27"	F4	Límite	Válido	Hoy, detecta si se rechaza “hoy” por error
10	fecha	"2026-10-28"	F5	Límite	Error fecha futura	Primer día inválido después de hoy
11	fecha	"27/10/2026"	F2	Inválido	Error de formato	Formato incorrecto
12	fecha	"2026-02-30"	F3	Inválido	Error de formato	Fecha inexistente
13	lugar	""	-	Inválido	Falta lugar	Campo vacío
14	observador	""	-	Inválido	Falta observador	Campo vacío
15	fecha	""	F1	Inválido	Falta fecha	Campo obligatorio

3. ¿Por qué cada nivel?

El proyecto incluye pruebas unitarias y de integración. Las unitarias verifican el comportamiento de una clase de forma aislada, mientras que las de integración comprueban que varios componentes funcionen correctamente al trabajar juntos.

Por ejemplo, **fechaDeMananaEsRechazad**a es una prueba unitaria porque solo valida la regla de negocio que impide registrar fechas futuras, sin necesidad de usar HTTP ni una base de datos. De forma similar, **registrarAvistamientoInvalidoResponde400YNoLoGuarda** también es unitaria, ya que verifica que el controlador no intente guardar la información cuando la validación falla.

En cambio, **registrarAvistamientoValidoResponde201YQuedaGuardado** es una prueba de integración porque comprueba el funcionamiento conjunto de la API. Esta prueba puede detectar errores que una prueba unitaria del validador no encuentra, como problemas al convertir el JSON recibido, fallos en la comunicación entre el controlador y el validador o errores al guardar y recuperar los datos desde la base de datos.

Estos errores ocurren en la frontera entre los componentes, que es justamente lo que las pruebas unitarias no ven porque reemplazan las piezas reales por dobles.

4. Los dobles

Para las pruebas utilicé dos dobles: **RelojFijo** como stub y un mock del repositorio creado con Mockito. Elegí un stub para el reloj porque el validador solo necesita que le devuelvan un valor (la fecha de hoy) y la prueba revisa el resultado, no cómo se llamó al reloj. Así las pruebas de fecha siempre producen el mismo resultado. Para el repositorio elegí un mock y no un fake, porque lo que quería comprobar era la interacción: que save se llame una vez si los datos son válidos y nunca si son inválidos. Un fake en memoria habría requerido implementar el repositorio completo. Sin estos dobles, las pruebas dependerían de la fecha real del sistema y de la base de datos, y serían más lentas y menos repetibles. Por ejemplo, la prueba **fechaDeMananaEsRechazada** usa la fecha 2026-10-28: sin el stub pasaría hoy, pero a partir del 29 de octubre esa fecha ya sería pasado y la prueba empezaría a fallar sin que nadie cambiara el código.

5. Lo que no probé

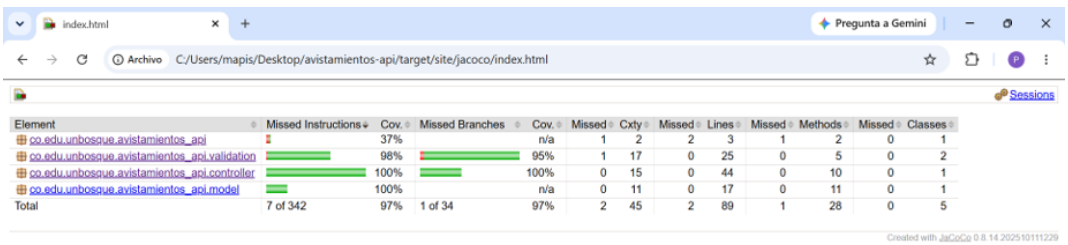
No todas las funcionalidades requieren pruebas específicas. Algunas pruebas decidí no realizarlas porque aportarían poco valor.

- **Getters y setters de Avistamiento:** No tienen lógica de negocio, por lo que una prueba solo comprobaría que un valor asignado es devuelto correctamente. Además, estos métodos ya se ejercitan de forma indirecta en las demás pruebas.

- **Peticiones con un id no numérico o con un JSON mal formado:** En estos casos es Spring quien responde automáticamente con un error antes de llegar al controlador. Probar estos escenarios significaría probar el framework y no la lógica propia de la API.

6. Lectura de la cobertura

La cobertura se midió con JaCoCo 0.8.14, ejecutando mvn verify, que corre las 34 pruebas (unitarias y de integración) y genera el reporte en target/site/jacoco/index.html.



6.1. Lo que quedó sin cubrir y la decisión tomada

- **Línea 51 de ValidadorDeAvistamiento: if (a.getLugar().trim().length() > 100) (rama amarilla: solo se probó el caso falso):** Ninguna prueba envía un lugar con más de 100

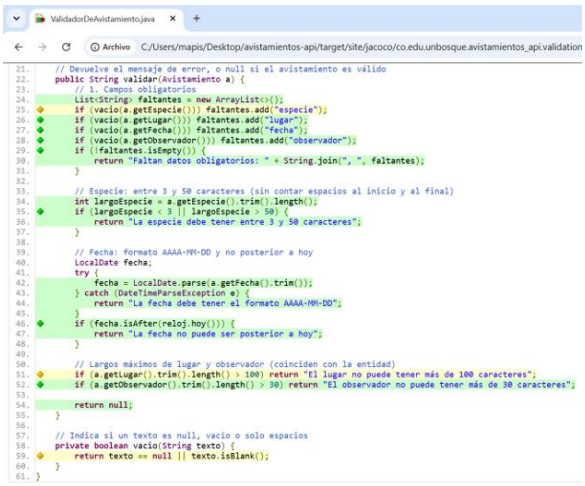
caracteres. Se decidió no agregar esa prueba porque este límite no hace parte de las reglas de negocio principales del taller, sino que corresponde a una restricción técnica alineada con el tamaño de la columna en la base de datos. Además, la misma lógica ya se valida en el campo observador cuando supera su longitud máxima, por lo que sería una prueba muy similar y con poco valor adicional.

- **Método main de AvistamientosApiApplication (por eso el paquete raíz tiene 37%):** También decidí no probarlo. Este método únicamente inicia la aplicación mediante SpringApplication.run(...) y no contiene lógica propia. Su funcionamiento ya queda cubierto por las pruebas de integración, que levantan la aplicación completa y verifican que la API arranque y funcione correctamente.

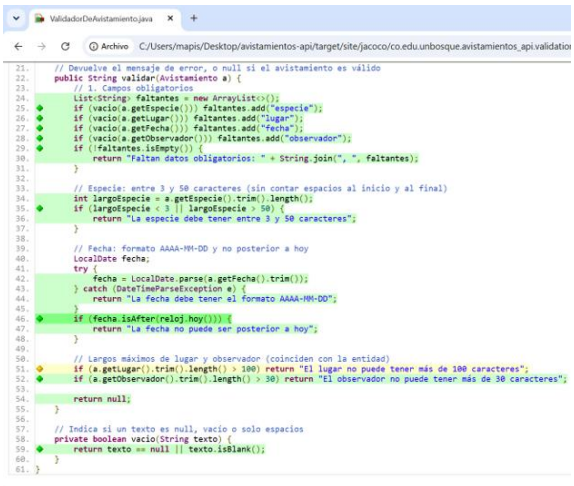
6.2. Conclusión del reporte

El reporte mostró que la cobertura es útil para identificar casos que aún no han sido ejecutados, pero no garantiza que las pruebas sean suficientes o de calidad. El criterio de cobertura cambia el resultado: en la primera medición, la cobertura de líneas mostraba todas las líneas ejecutadas, pero la cobertura de ramas mostró 3 ramas sin probar. Dos de ellas (líneas 25 y 59) correspondían a entradas vacías o con solo espacios, y quedaron cubiertas al implementar los casos 6, 13 y 14 de la tabla. La tercera (línea 51) es la que decidí no probar (sección 6.1).

Primera cobertura de pruebas:



Ramas cubiertas después de implementación:



Sin embargo, una cobertura alta no asegura que no existan defectos. Lo realmente importante es elegir buenos casos de prueba. Por esa razón, los escenarios se diseñaron utilizando partición de equivalencia y valores límite, mientras que la cobertura se utilizó como una herramienta para encontrar posibles vacíos en las pruebas.